

Project Requirements

Completion requirements

Project Requirements: Cross-Platform Mobile Development

1. Project Overview and Practical Scope

As part of this course, you will work in teams of two to design and develop a fully functioning mobile application using one of the following cross-platform frameworks:

- **Flutter** (Dart)
- **.NET MAUI** (C#/.NET)
- **React Native** (JavaScript/TypeScript)

Your app should solve a real problem for a defined target group and showcase the core capabilities of a modern cross-platform mobile application. You are free to choose the topic of your app. However, the project must be technically meaningful, realistically implementable within the semester, and complex enough to demonstrate your understanding of the following required aspects:

Required functional aspects:

- **Navigation:** The app must include at least three distinct screens / views and support structured navigation (e.g., stack navigation, tabs, deep links, etc.).
- **State Management:** The app must manage and update state in a controlled, consistent way (local state and/or shared/global state).
- **API Integration:** The app must communicate with at least one external data source (e.g., REST API) and display dynamic data.
- **Native Feature:** The app must use at least one device capability (e.g., camera, location/GPS, push notifications, file system, sensors, etc.).
- **Offline / Persistence:** The app must store data locally (e.g., cached API data, user preferences, saved items) so that the core experience works without an internet connection.
- **Testing / Stability:** The app must include basic tests (e.g., unit test, component test, or equivalent) and run stably on at least one real device or emulator.

In addition:

- You are expected to follow a clean project structure and a modern architectural pattern (e.g., MVVM, Provider/BLoC, Hooks/Context, etc.).

- You will work with our GitHub Organization (**FHTW-CPD**). Each team is required to generate its own team repository using the official template repository link provided on Moodle and commit regularly.
- **AI Tools:** The use of AI coding assistants (GitHub Copilot, ChatGPT, Claude, Cursor, etc.) is permitted according to our **AI Usage Policy**. However, you remain 100% accountable for explaining and defending every part of your codebase during the final review.
- At the end of the semester, you will present your application and walk through your code.

This is not just a “hello world” exercise. The [project](#) is meant to simulate a realistic small mobile product.

Example [project](#) ideas (you are not limited to these):

- A personal habit tracker with reminders and offline history
- A recipe finder that fetches data from an external API and lets users save favorites locally
- A “Campus Helper” app with map/location features and cached info for offline use
- A travel packing / checklist app with photo capture and persistent stored lists
- A workout / activity logger using sensors or GPS
- A shared shopping list app for two users (simple sync via API, local cache when offline)

Your app should have a clear purpose, identifiable user roles (if relevant), and a usable interface. Visual polish matters, but functional clarity matters more.

2. [Project](#) Specification Document (Mandatory deliverable before development)

Your first major task is to create a [Project](#) Specification Document. This document defines your concept before you build it and must be submitted before LV3. This specification is part of the formal grading.

The [Project](#) Specification must be a PDF (exported from Word / Google Docs / Markdown). Please use the following sections and headings:

- **2.1 [Project Introduction](#)**
 - Team member names
 - Working title of your app
 - Short summary of what the app does and why it exists

- Target users and the specific problem you are solving
- **2.2 Chosen Framework**
 - Framework choice (Flutter / .NET MAUI / React Native)
 - Brief explanation for your choice (e.g., language experience, ecosystem, performance, deployment targets)
 - Target platforms (Android required, iOS optional if you have access to macOS / Xcode)
- **2.3 Planned Features (User Stories)**
 - Define core features using user stories: *“As a [type of user], I want to [do something], so that [I achieve a goal].”*
 - Provide at least 8–12 user stories covering: navigation flows, data fetching (API usage), state changes/interactions, native feature usage, and offline/persistence behavior.
 - Each story must include acceptance criteria (“This is done when...”).
- **2.4 Screen / Navigation Structure**
 - List main screens (e.g., Home, Details, Settings, CameraScreen) and briefly describe what is shown, navigation flows, and required internal state.
- **2.5 API and Data Sources**
 - Describe external API(s) or data source(s), fetched data, screen mapping, authentication requirements, or mock data fallback plans.
- **2.6 Native Feature Usage**
 - Explain which native capability you will integrate, where it is used, and why it is meaningful for your use case.
- **2.7 Offline / Persistence Strategy**
 - Detail what data will be stored locally, why, and how/when data syncs or updates.
- **2.8 Technical Approach & Architecture**
 - Outline codebase structure (e.g., View/ViewModel, Screens + hooks, Widgets + state management, folder organization).
- **2.9 Changelog**

- Document any changes made later in the semester (what changed and why) to maintain traceability between your original plan and final build.

Important: Late or missing specifications will negatively impact your overall [project](#) grade.

3. Final [Project](#) Submission and Code Review

At the end of the semester, you will submit:

1. Your full [project](#) (via your team's public GitHub repository generated from the course template, submitting the URL to Moodle).
2. Your updated [Project](#) Specification PDF (including changelog).
3. Participation in a short [project](#) review / oral check.

By this point, your app should be stable, functional, and aligned with all required technical aspects.

What happens in the final review?

- Live demo of your running app on an emulator or physical device.
- Walkthrough of relevant code sections.
- Explanation of architecture, state management, and design decisions.
- Reflection on achievements, challenges, and potential improvements.

How the [project](#) is graded: Your [project](#) will be evaluated across six categories (10 points each → total: 60 points):

1. **Navigation & App Structure (10 points):** Clear screen flow, consistent navigation patterns, logical component separation, usability.
2. **UI / State Management (10 points):** Clean, consistent UI, proper handling of local/shared state, reactivity, responsive layouts.
3. **API Integration / Data Flow (10 points):** Reliable data fetching, error and loading handling, clean service/API layer.
4. **Native Feature Integration (10 points):** Meaningful native capability implementation, correct permission handling, fully functional execution.
5. **Offline & Persistence (10 points):** Data availability without network, clean local storage implementation, graceful online/offline handling.
6. **Code Quality & Presentation (10 points):** [Project](#) structure, readability, GitHub commit quality, compliance with the course **AI Usage Policy**, basic testing, and the ability to explain and defend your own code in the review.

Important rule for passing the course: The [project](#) grade must be positive. Failing the [project](#) results in failing the course, regardless of points earned elsewhere.

Traceability requirement: Upon final submission, you must update your [Project](#) Specification PDF so that each core user story is explicitly linked to its implementation in code (e.g., “As a user I want to take a photo...” → *Implemented in screens/CameraScreen.jsx and services/MediaService.js*). This is mandatory for a passing grade.